



Embedded System Lab Manual

LAB EXPERIMENT NO. 10

Timer Base STM32 Programming with STM32F103C8T6 Blue Pill C++

Submitted To:

Sir Yasir

Submitter by:

Ahsan Basharat

Date of Submission:

10/06/2026

Introduction

In this laboratory experiment, we learned about timers in the STM32 microcontroller using STM32CubeIDE and the STM32 HAL (Hardware Abstraction Layer) Library. Timers are one of the most important peripherals in embedded systems because they provide accurate timing and counting functions independent of the CPU. They are widely used for generating precise delays, measuring time intervals, creating periodic events, producing PWM signals, counting external pulses, and triggering other peripherals such as the ADC.

In this laboratory, we learned how to configure and use the TIM3 timer of the STM32F103C8T6 Blue Pill microcontroller using STM32CubeMX, STM32CubeIDE, and the STM32 HAL (Hardware Abstraction Layer) Library. The timer was initialized through CubeMX and controlled in C++ using a custom Timer HAL class.

What is Timer ?

A timer is a hardware peripheral inside a microcontroller that counts clock pulses. It is used to measure time, generate delays, create periodic events, count external signals, and generate PWM signals without continuously using the CPU.

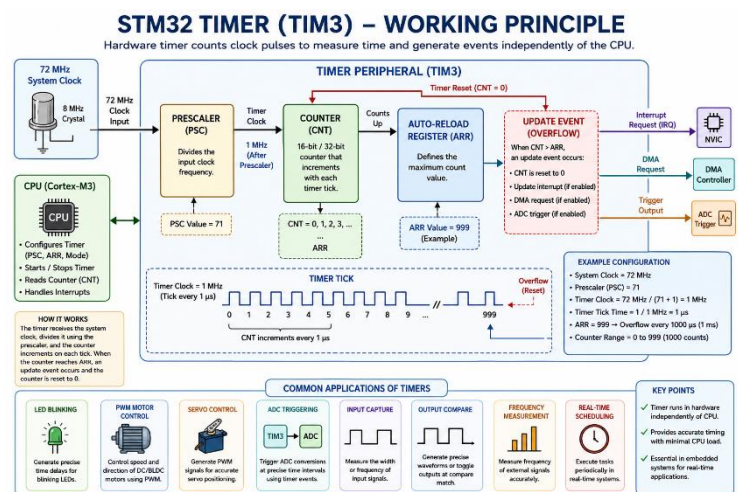
The Role of Timers

Timers in embedded systems are used to measure time intervals, control delays, and trigger events at regular intervals. They are essential for managing time-based tasks such as:

Pulse Width Modulation (PWM): Used for controlling motors, dimming LEDs, and generating analog signals from digital data.

Event Timing: Measuring how long a specific event lasts, like a button press or a sensor reading.

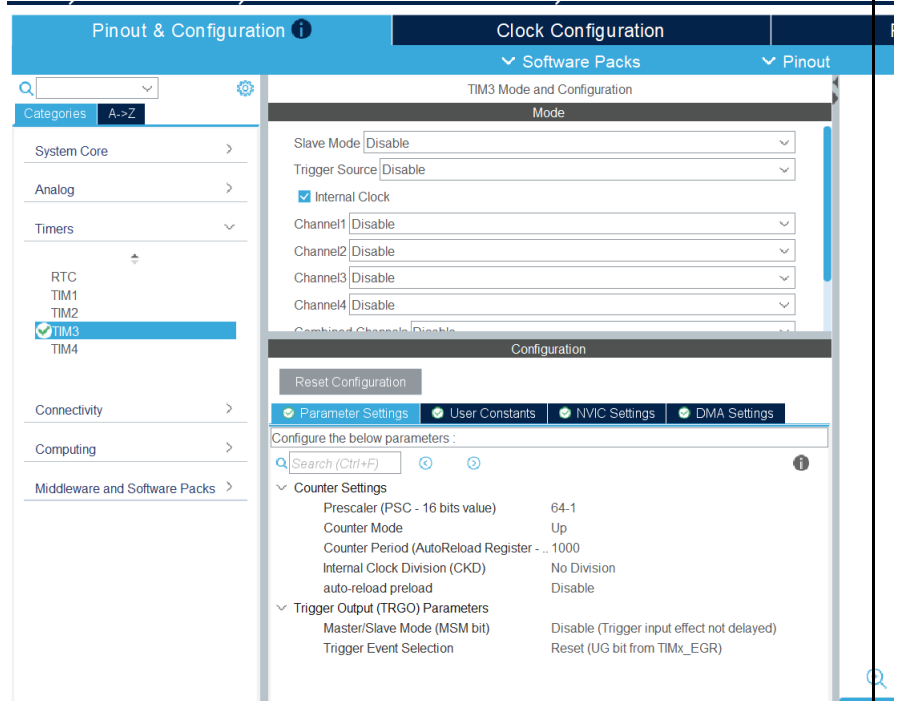
Task Scheduling: In real-time systems, timers help ensure tasks are executed at precise intervals.



STM32CUMEMX TIMER Setup

In this section we discuss the basic setup of timer.

- TIM3 is configured as a 16-bit general-purpose timer to measure time and generate periodic events.
- Internal Clock is enabled, allowing TIM3 to use the microcontroller's internal clock as its timing source.
- Prescaler (PSC = 63) divides the 64 MHz system clock to 1 MHz, so the timer increments every 1 μ s.
- Counter Mode is set to Up, and the Auto-Reload Register (ARR = 1000) causes the timer to count from 0 to 1000 before overflowing and restarting.
- All timer channels, Slave Mode, and Trigger Source are disabled, as the timer is used only as a basic hardware timer for measuring elapsed time in this experiment.



HAL_TIME Libraries

HAL_TIM_Base_Init()

This function initializes the timer peripheral according to the configuration stored in the TIM_HandleTypeDef structure. It prepares the timer for operation but does not start the timer.

Prototype

```
HAL_StatusTypeDef HAL_TIM_Base_Init(TIM_HandleTypeDef *htim);
```

Htim Pointer to the timer handle (TIM_HandleTypeDef) containing the timer configuration.

CODE STRUCTURE

- The constructor saves the timer peripheral (TIM3) and its HAL handle (htim) into the class variables and automatically calls init() to initialize the timer.
- It checks whether TIM3 has already been initialized using is_init[2]. If it is already initialized, the function returns immediately to avoid reinitialization.
- If TIM3 is not initialized, is_init[2] is set to true and MX_TIM3_Init() is called to configure the timer with the settings generated by STM32CubeMX.
- When the object is destroyed, is_init[2] is reset to false, indicating that TIM3 is no longer initialized or in use.
- HAL_TIM_Base_MspDeInit(_htim) deinitializes the timer hardware by disabling the timer clock and releasing any associated hardware resources, allowing the timer to be initialized again later if needed.

```
class Time
{
public:
    Time(TIM_TypeDef *Instance, TIM_HandleTypeDef *htim);
    virtual ~Time();

    virtual HAL_StatusTypeDef start(void);
    virtual HAL_StatusTypeDef stop(void);

    uint32_t read(void);
    virtual void reset(void);

protected:
    TIM_TypeDef *_Instance;
    TIM_HandleTypeDef *_htim;

private:
    static bool is_init[TIME_COUNT];
    void init(void);
};

Time::Time(TIM_TypeDef *Instance, TIM_HandleTypeDef *htim)
: _Instance(Instance), _htim(htim)
{
    init();
}

Time::~Time()
{
    if (_Instance == TIM3)
    {
        is_init[2] = false;
        HAL_TIM_Base_MspDeInit(_htim);
    }
}

void Time::init(void)
{
    if (_Instance == TIM3)
    {
        if (is_init[2])
        {
            return;
        }

        is_init[2] = true;
        MX_TIM3_Init();
    }
}
```

HAL_TIM_Base_DeInit()

This function deinitializes the timer and restores it to its default reset state.

Prototype

```
HAL_StatusTypeDef HAL_TIM_Base_DeInit(TIM_HandleTypeDef *htim);
```

HAL_TIM_Base_Start()

Starts the timer in Polling Mode. The timer begins counting immediately.

Prototype

```
HAL_StatusTypeDef HAL_TIM_Base_Start(TIM_HandleTypeDef *htim);
```

__HAL_TIM_SET_COUNTER()

This can Sets the timer counter (CNT register) to a user-defined value.

Prototype

```
__HAL_TIM_SET_COUNTER(htim, Counter);
```

- Htim->Pointer to the timer handle.
- Countee->New counter value.

__HAL_TIM_GET_COUNTER()

This can Reads the current timer counter value from the CNT register.

Prototype

```
__HAL_TIM_GET_COUNTER(htim);
```

CODE STRUCTURE

- The start() function starts the timer by calling HAL_TIM_Base_Start(_htim). Once started, the timer counter begins counting according to the configured clock frequency and prescaler. It returns a HAL_StatusTypeDef value indicating whether the operation was successful.
- The stop() function stops the timer by calling HAL_TIM_Base_Stop(_htim). This disables the timer counter while preserving its current value, allowing the timer to be restarted later without losing the count.
- The reset() function resets the timer counter to zero using the __HAL_TIM_SET_COUNTER(_htim, 0) macro. This is useful when a new timing measurement is required without reinitializing the timer.

```
class Time
{
public:
    Time(TIM_TypeDef *Instance, TIM_HandleTypeDef *htim);
    virtual ~Time();

    virtual HAL_StatusTypeDef start(void);
    virtual HAL_StatusTypeDef stop(void);

    uint32_t read(void);
    virtual void reset(void);

protected:
    TIM_TypeDef *_Instance;
    TIM_HandleTypeDef *_htim;

private:
    static bool is_init[TIME_COUNT];
    void init(void);
};

1
2 HAL_StatusTypeDef Time::start(void)
3 {
4     return HAL_TIM_Base_Start(_htim);
5 }
6
7 HAL_StatusTypeDef Time::stop(void)
8 {
9     return HAL_TIM_Base_Stop(_htim);
10 }
11
12 void Time::reset(void)
13 {
14     __HAL_TIM_SET_COUNTER(_htim, 0);
15 }
16
17 uint32_t Time::read(void)
18 {
19     return __HAL_TIM_GET_COUNTER(_htim);
20 }
```

- The read() function reads and returns the current value of the timer counter using the `__HAL_TIM_GET_COUNTER(_htim)` macro. The returned `uint32_t` value represents the number of timer ticks that have elapsed since the timer started or was last reset.

Main.cpp

- A Time object named `tim3` is created using the TIM3 peripheral and its corresponding handle (`htim3`). This object provides an object-oriented interface for controlling the hardware timer.
- The ADC_DMA object and its channel configuration are commented out, indicating that the focus of this program is on testing the timer functionality.
- The `tim3.start()` function is called to start the TIM3 hardware timer. Once started, the timer begins counting based on the configured timer clock and prescaler settings.
- Two variables, `tick_us_previous` and `tick_us_now`, are initialized to zero. These variables store the previous and current timer counter values for measuring elapsed time.
- Inside the infinite `while(1)` loop, the current timer count is read using `tim3.read()` and stored in `tick_us_now`.
- The program calculates the elapsed timer counts by subtracting `tick_us_previous` from `tick_us_now`.
- The condition `if ((tick_us_now - tick_us_previous) >= 1000)` checks whether 1000 timer counts have elapsed. If the timer is configured to increment every 1 μ s, this corresponds to an elapsed time of 1 ms.
- When the condition is satisfied, `tick_us_previous` is updated with the current timer count. This resets the reference point for measuring the next 1 ms interval.

```

1  Time tim3(TIM3, &htim3);
2
3  // ADC DMA Object
4  // ADC_DMA adc(ADC1, &hadc1, 100);
5
6  // adc.add_channels(ADC_CHANNEL_3);
7
8  // Start peripherals
9  tim3.start();
0  adc.start();
1
2  uint32_t tick_us_previous = 0;
3  uint32_t tick_us_now = 0;
4
5  while (1)
6  {
7      tick_us_now = tim3.read();
8
9      if ((tick_us_now - tick_us_previous) >= 1000)
0      {
1          tick_us_previous = tick_us_now;
2
3      }
4  }
5  }

```